

RGB-D Object Pose via HSV Detection and Open3D Point Cloud Localization

Noel Elias · object-pose-estimation · MIT License

Abstract. This repo benchmarks classical **RGB-D** colored-cube **3D position** using **OpenCV** HSV detection and a masked **Open3D** point-cloud centroid, with millimeter error scored against simulator ground truth in **NumPy**. Scenario evals run under clean, RGB noise, depth noise, and occlusion conditions (~**0.5 mm** mean error on clean frames). **ROS2 Humble** integration publishes live pose over **TF**, visualizes in **RViz**, includes an **rcldcpp** C++ latency monitor, and ships as a **Docker** demo. **Colab-ready** notebooks walk through setup, the pipeline, evals, and ROS2, using RGB-D frames from **Isaac Sim**, **Gazebo**, or bundled synthetic exports.

1. Problem & motivation

Classical RGB-D baseline for a colored cube: how far HSV + Open3D point-cloud centroid gets with simulator ground truth, and how it maps to ROS2 — without a learned detector.

2. Method (summary)

Stage	Description
Detection	OpenCV HSV + largest contour → segmentation mask — rgbd_pose/detection.py
Geometry	data/geometry.py — CameraIntrinsics, Pose3D
3D pose	Masked depth → Open3D point cloud → outlier filter → optical-frame centroid — rgbd_pose/pose.py
Eval / viz	evals/metrics.py — scenario metrics + detection overlays
Ground truth	Export meta.json or data/synthetic.py
Data priority	Isaac exports → Gazebo exports → synthetic_exports/
Export	notebooks/data.ipynb — auto-pick Isaac/Gazebo/synthetic, export_sim_frames()
Gazebo scene	assets/cube/cube_rgbd.sdf — headless RGB-D via data/gazebo.py
Load	data/frame.py — load_frame(), describe_frame_sources()
Eval scenarios	clean, rgb_noise ($\sigma \approx 15$), depth_noise (≈ 20 mm), occlusion band
ROS2	rgbd_pose_node.py → /object_pose + TF; C++ latency monitor; RViz demo (--viz)

3. Results

3.1 Perception evals (20 frames per scenario, synthetic)

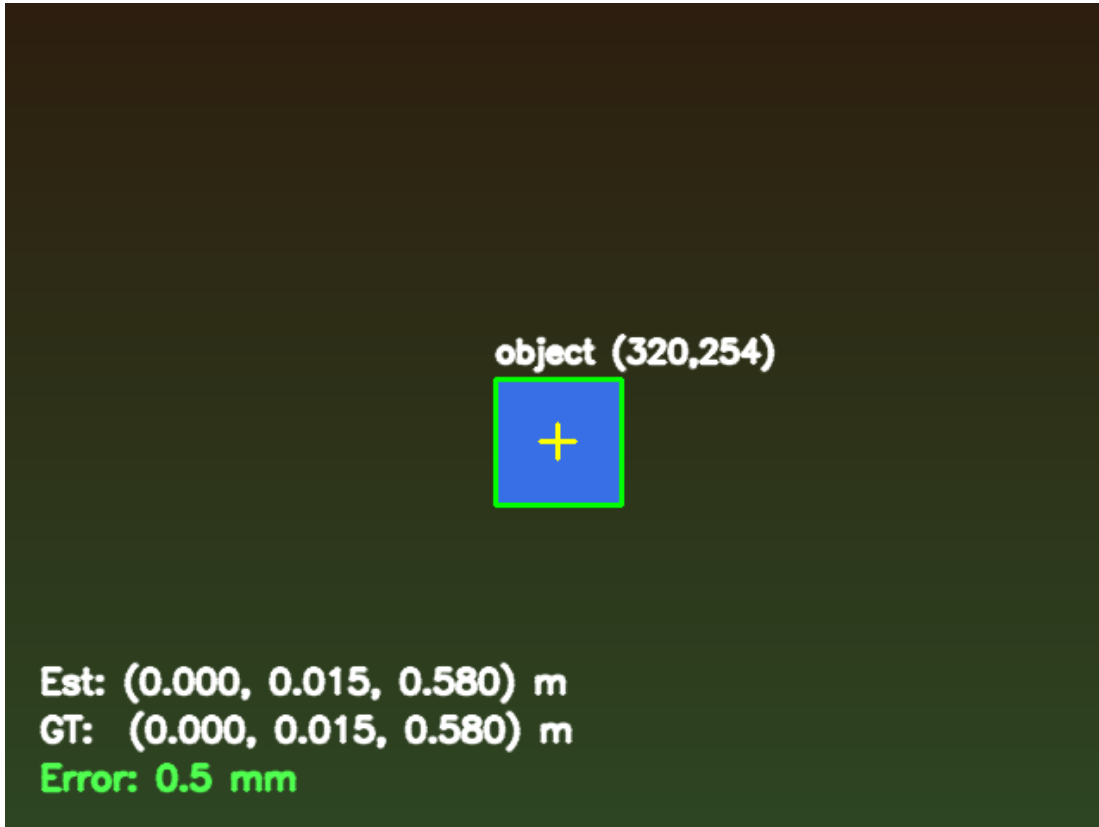
Scenario	Detection	Mean error (mm)	FPS
clean	100%	0.5	286
rgb_noise	100%	0.7	274
depth_noise	100%	1.0	266
occlusion	0%	-	1783

3.2 ROS2 pipeline (Docker demo)

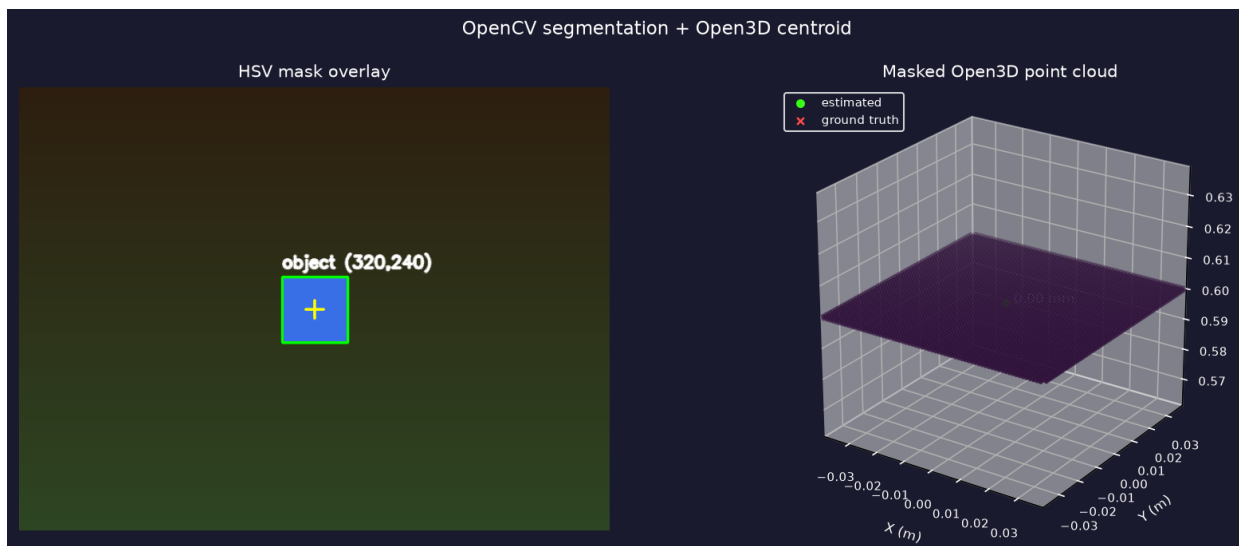
Component	Role
demo_rgbd_publisher.py	Animated synthetic RGB-D at 10 Hz
rgb_d_pose_node.py	ObjectDetector + PoseEstimator → /object_pose + TF
pose_latency_monitor (node)	C++ subscriber: stamp latency + FPS
Run (headless)	./ros2/run_demo.sh

Run (RViz GIF) ./ros2/run_demo.sh --viz → assets/demo/ros2_rviz_demo.gif
Notebooks setup → pipeline → evals → ros2

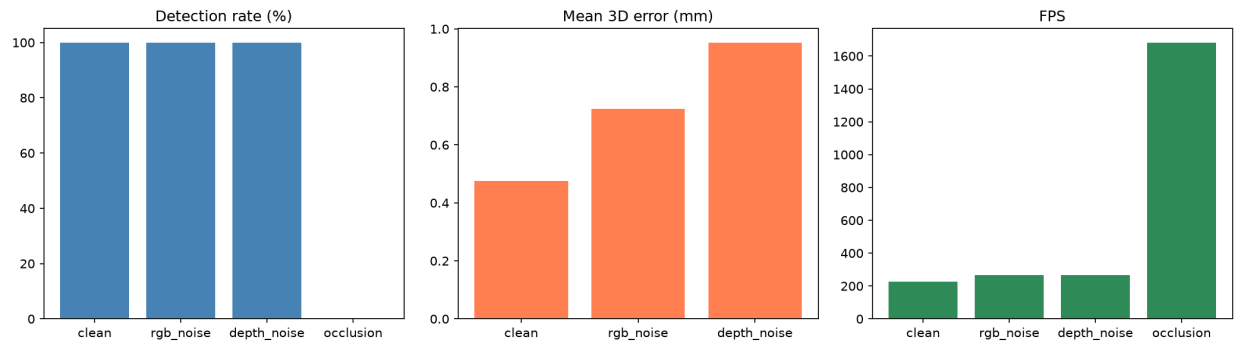
3.3 Qualitative outputs



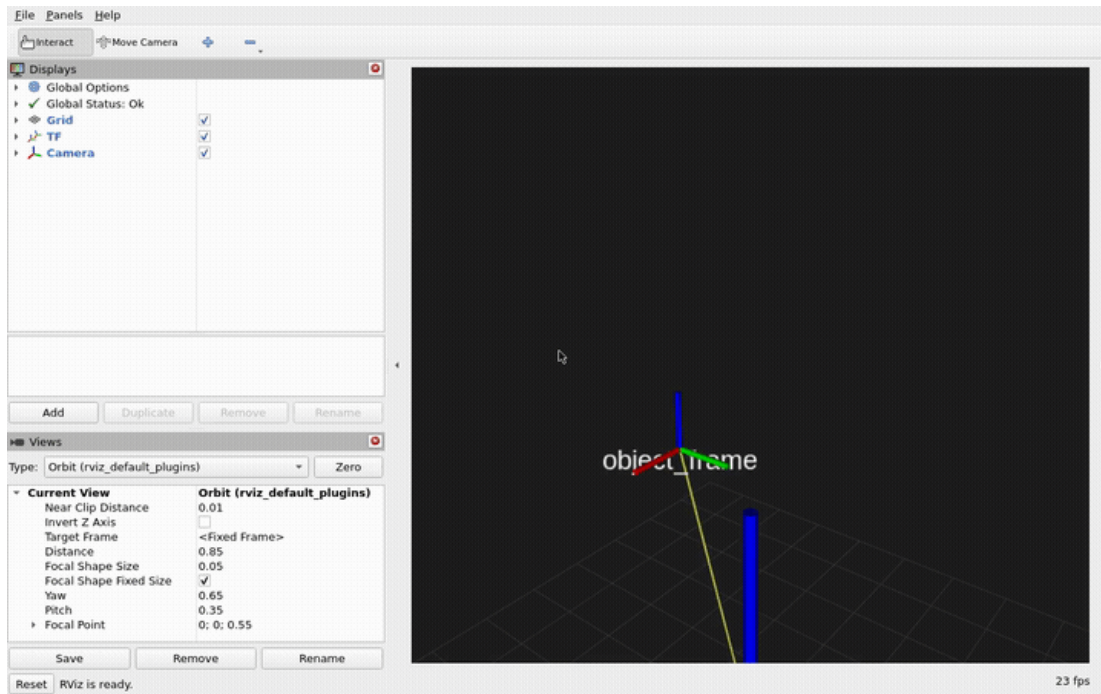
Pipeline overlay on moving synthetic frames



HSV mask overlay and zoomed Open3D point cloud (centroid vs ground truth)



Eval summary



ROS2 RViz: object_frame tracks detected cube (./ros2/run_demo.sh --viz)

4. Simulation export

Isaac Sim (GPU, `data/isaac.py`) and Gazebo (CPU, `data/gazebo.py` + `assets/cube/cube_rgbd.sdf`) via `notebooks/data.ipynb` (auto-picks backend). Bundled demo frames live in `data/synthetic_exports/`; `isaac_exports/` and `gazebo_exports/` hold real sim output when exported locally. Colab runs perception only — upload exports to Drive.

5. Limitations & next steps

- HSV detector is lighting-sensitive; real sim export validates sim-to-perception gap.
- Position-only pose (3D centroid); no 6D orientation yet.
- On-robot latency will exceed the in-container stamp delay; TF/RViz demo uses synthetic RGB-D.